Proceedings of the 43rd Chinese Control Conference
July 28-31, 2024, Kunming, China

# Autonomous Maneuver Decision for Unmanned Fighters in Multi-Agent BVR Air Combat

Fuwei Tan[1], Xiaohui Sun[2], Zhen Zhu[1], Qing-Hao Zhang[1], Yongning He[2], Jun-Guo Lu[1]

1. Department of Automation, Shanghai Jiao Tong University, Shanghai 200240, P. R. China
E-mail: wwddv1995@sjtu.edu.cn; dyzz0928@sjtu.edu.cn; zhangqinghao@sjtu.edu.cn; jglu@sjtu.edu.cn

2. Shanghai Aerospace Control Technology Institute, Shanghai 201108, P. R. China
E-mail: huizi123717@163.com; 15611776036@163.com

**Abstract:** In this paper, the issue of autonomous maneuver decision-making in multi-agent unmanned fighter air combat including missile evasion is considered. Firstly, the model of multi-agent unmanned fighter air combat is established by reducing dimensions. The reward function of the MATD3 algorithm model suitable for air combat is designed to realize multi-agent air combat maneuver decisions. Considering the instability in the training process of the MATD3 algorithm in addressing multi-agent continuous action space problems, especially in BVR air combat, an improved multi-agent selective experience replay method is proposed to enhance training stability. Finally, the multi-agent air combat battles are simulated on DCS World and the effectiveness of the MBMATD3 algorithm with the improved experience replay method is demonstrated.

**Key Words:** Deep reinforcement learning, MATD3 algorithm, BVR air combat, multi-agent, missile evasion, maneuver, DCS World

## 1 Introduction

With the development of autonomous navigation, remote flight control, and the aviation industry, unmanned combat aerial vehicles (UCAVs) have been widely used by military forces worldwide. Compared to manned combat aerial vehicles, unmanned combat aerial vehicles have more advantages in fields of covert operations, surveillance missions, and long-duration combat missions [1, 2]. However, current UCAVs rely more on manual control by operators and remote communication technology. This results in communication control delays and makes UCAVs more susceptible to electronic warfare interference and suppression [3]. Therefore, a multi-agent autonomous decision-making model that enables UCAVs to make autonomous intelligent decisions in combat missions is currently a hotspot of research in the aviation field.

A multi-agent autonomous decision-making model is able to circumvent remote control by human operators and remote communication technology. Instead, it autonomously learns and generates intelligent decisions for air combat evasion [4]. It is also able to make collaborative decisions within multiple intelligent UCAV agents [5]. Methods for autonomous air combat decision-making encompass various fields, including reinforcement learning, neural networks, game theory optimization algorithms, etc. In [6, 7], game theory methods are utilized to determine the optimal tactics for flight control and strategies in beyond-visual-range (BVR) air combat scenarios involving multiple UAVs. In [8, 9], genetic algorithms and deep learning methods are employed to address the challenge of air combat decision-making in scenarios beyond line-of-sight. However, continuous action space problems like air combat involve complex strategy and action search spaces. Genetic algorithms are not as efficient as reinforcement learning algorithms in solving such problems. In [10], an expert system is developed to address the tactical decision problem in air combat. Since expert systems rely more on prior knowledge and rules, deep reinforcement learning algorithms are more suitable for addressing air combat problems with complex uncertainty. In [11], a deep neural network is designed for making decisions regarding air-offensive campaigns. In [12], a decision-making model for Unmanned Combat Aerial Vehicles in air combat is constructed by incorporating a reinforcement learning-enhanced tactical pursuit point. Alternatively, deep reinforcement learning methods have been thoroughly investigated and applied, showcasing outstanding performance in the realm of air combat in recent years [13–15]. Deep deterministic reinforcement learning algorithms including DDPG and TD3 are well-suited for problems with continuous action spaces in air combat [16]. However, in realistic multi-agent air combat missions, destroyed units and changes in dimensions of observations lead to instability and low performance on deep reinforcement learning methods such as MADDPG [17].

Inspired by the above, the improved MATD3 algorithm with multi-agent selective experience replay method is proposed in this paper. All contributions in this paper are shown as follows:

i) In this paper, factors such as missiles and evasion, coordinated combat of multiple fighters, as well as the destruction of enemies and allies are considered. These factors collectively determine the result of the simulation battle and increase the complexity of the strategy space.

ii) To effectively extract battlefield information features, raw data of fighters is converted into visual angles closer to a first-person perspective of the pilot. A new form of reward function based on five important parameters is designed, allowing agents to improve the search efficiency of strategy in air combat simulation battles.

iii) Compared with the original MATD3 algorithm, the MATD3 algorithm with multi-agent selective experience re-

This work is supported by "the Fundamental Research Funds for the Central Universities" (23GFC-JJ03-214).

This work is also supported by the joint funds of Industry-university-research of Shanghai Academy of Spaceflight Technology, China Aerospace Science and Technology Corporation (USCAST2022-34).

The work is also supported by the National Natural Science Foundation of China (No. 62073217 and No. 62303309).



Authorized licensed use limited to: ULAKBIM UASL - YILDIZ TEKNIK UNIVERSITESI. Downloaded on November 05,2026 at 10:37:09 UTC from IEEE Xplore. Restrictions apply.

play method (MBMATD3) enhances more training stability and efficiency in air combat scenarios containing destroyed units, which allows better decision-making in realistic multi-agent air combat missions and higher winning of air combat simulation battles.

## 2 MATD3 Algorithm

One prominent issue with the DDPG algorithm is its instability during training [18]. For reinforcement learning algorithms suitable for discrete space problems, researchers have proposed various solutions to address the problem of overestimating bias. In practice, the interaction between the agent and the environment introduces sampling errors in the information, thereby causing a certain level of error in the Q-values. As a result, the maximum value of Q-values with errors in a real-world scenario will be greater than the maximum value of the true Q-values, as shown in:

$$E_{\epsilon}[\max_{a'} Q(s', a') + \epsilon] > \max_{a'} Q(s', a'), \tag{1}$$

where $\epsilon$ is the error in the Q-values, $s'$ and $a'$ are state and action vectors in the environment.

This results in the accumulation of errors, even if they initially have a mean of 0, through the Bellman equation, leading to highly biased estimates in the end. Moreover, during the process of function iteration approximation, bias is unavoidable. Therefore, for DDPG algorithms applicable to continuous spaces, eliminating this bias overestimation is the key to approaching the optimal policy. Additionally, inaccurate estimates of the Q-function can affect the direction of policy updates in the algorithm. The algorithm may assign higher values to policies other than the optimal policy and update the policy towards those suboptimal policies during policy updates, thereby affecting the overall training effectiveness.

To address the issue of Q-function overestimation in the DDPG algorithm, the TD3 algorithm is built upon the ideas from Double Q-learning [19]. It introduces a novel variant of the dual Q-network to replace the original critic network in the actor-critic algorithm. TD3 algorithm adopts a method that takes the lower bound of the dual Q-network, as shown in:

$$y_1 = r + \gamma \min_{i=1,2} Q_{\theta_i'}(s', \pi_{\phi_i}(s')), \tag{2}$$

where $y_1$ represents value of target $Q$ network, $r$ represents the reward from the environment, and $\theta$ is the parameter of the actor network under strategy $\pi$.

The critic and actor optimization objective function is updated as follows:

$$\theta \leftarrow argmin_{\theta} N^{-1} \sum (y - Q_{\theta}(s, a))^2. \tag{3}$$

$$\nabla_{\phi} J = N^{-1} \sum \nabla_{\phi} \pi(s) \times \nabla_{aQ_{\theta_1}(s,a)|_{a=\pi(s)}}. \tag{4}$$

For multi-agent continuous action space scenarios, the MATD3 algorithm, an improved algorithm of TD3 algorithm for multi-agent scenarios, can effectively solve the issues of coordinating strategies among multiple agents [20]. MATD3 algorithm is suitable for multi-agent environments where each agent has its own actor and critic networks. The actor network of each agent outputs its action policy, and the critic networks of all agents receive action information from all agents as well as the environmental state to evaluate the value function of actions. By incorporating information about the policies of other agents, the MATD3 algorithm allows the actor network of each agent to better coordinate decisions, leading to improved overall system performance.

## 3 Problem Formulation and Model Design

### 3.1 Multi-Agent Air Combat Problem Formulation

In this paper, the following multi-agent air combat problem is considered: Both sides of fighters are the same type as four F/A-18C Hornet fighters. Each fighter follows a 6-degrees-of-freedom motion model, navigating through three-dimensional space to simulate the air combat environment.

In order to better describe the state of air combat problem involving fighter agents, the following parameter information is defined: the three-dimensional coordinates of our aircraft and the enemy aircraft, denoted as $x_a$, $y_a$, $z_a$ and $x_e$, $y_e$, $z_e$; the three-dimensional vector velocity components of our aircraft and the enemy aircraft, denoted as $v_{x_a}$, $v_{y_a}$, $v_{z_a}$ and $v_{x_e}$, $v_{y_e}$, $v_{z_e}$; attitude information for our aircraft, including pitch angle $\theta_p$, roll angle $\theta_b$, and yaw angle $\theta_y$; the three-dimensional coordinates of two enemy missiles, denoted as $x_{m_a}$, $y_{m_a}$, $z_{m_a}$ and $x_{m_e}$, $y_{m_e}$, $z_{m_e}$; and their absolute velocity vector magnitudes, $v_{m_a}$ and $v_{m_e}$. The state variables mentioned above are also the observation information of each fighter agent. Each agent achieves flight control via action variables including pitch, bank, and engine actions, which are denoted as $\alpha, \beta, engine$.

### 3.2 Modeling of Multi-Agent Air Combat

Modern fighter aircraft, especially advanced models like the F/A-18C Hornet, primarily employ onboard beyond-visual-range air-to-air missiles to engage enemy aircraft. These missiles are equipped with infrared guidance sensors, such as AIM-120C, allowing them to track heat-emitting targets on enemy aircraft. For each aircraft, converting raw data into visual angles closer to a first-person perspective of the pilot can effectively extract battlefield information features, making the aircraft agent explore the optimal strategy more effectively, as shown in Fig. 1.

After reducing the dimension of the three-dimensional air combat model along the XZ and XY planes, the two-dimensional air combat models are shown in Fig. 2 and Fig. 3.

In this paper, the environment state S consists of vectors $s = [v, h, pitch, bank, \phi, D, \phi_m, D_m, v_m]$, where $v$, $h$ represent the velocity and altitude of our fighter aircraft. $pitch, bank$ represents the pitch and yaw angle of our fighter aircraft. $\phi$ represents the sight angle vector from the enemy to our aircraft. $\phi_m$ represents the sight angle vector from the enemy missiles to our aircraft. $D, D_m$ represents the distance vector from our fighter aircraft to enemy fighter aircraft and missiles. $v_m$ represents the velocity of enemy missiles. Action A consists of vectors $a = [\alpha, \beta, engine]$, which represents the vector of the pitch, bank, and engine action of agents.


Authorized licensed use limited to: ULAKBIM UASL - YILDIZ TEKNIK UNIVERSITESI. Downloaded on November 05,2026 at 10:37:09 UTC from IEEE Xplore. Restrictions apply.

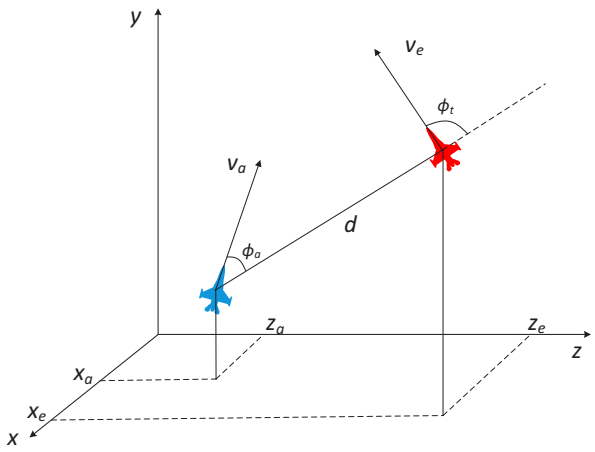



Fig. 1: 3-D air combat model

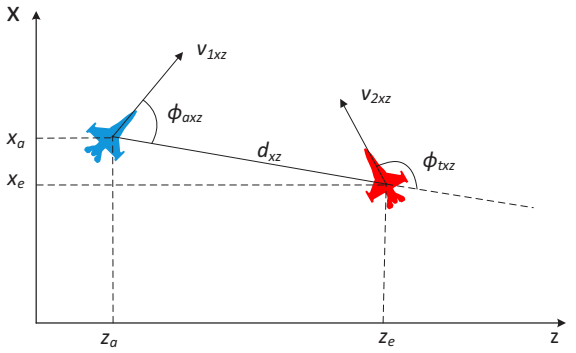



Fig. 2: 2-D air combat model on XZ plane

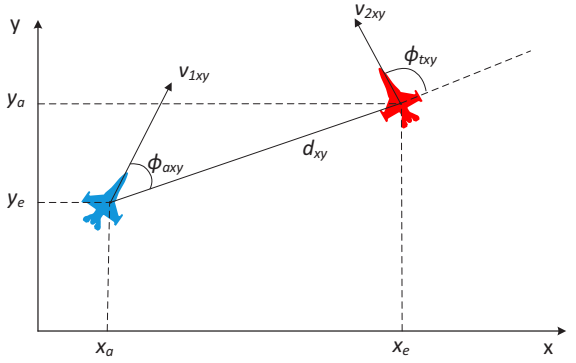



Fig. 3: 2-D air combat model on XY plane

## 4 Advancements in Autonomous Maneuver Decision

### 4.1 Automatic flight control

In order to enhance the flight control stability of our aircraft in air combat missions, the flight state variables of our aircraft are utilized instead of control stick input variables in this paper. After receiving the target flight state variables from the intelligent algorithm model output, our aircraft agent approaches its state variables to the target values using increment PID control.

### 4.2 MBMATD3 Algorithm

After inputting battlefield state variables observed by multiple intelligent agents of our aircraft into the algorithm network of different intelligent agents, the output state variables from different actors of the intelligent algorithm are deployed to the aircraft intelligent agents. The details of the MATD3 algorithm are shown in Table 1.

Table 1: Training Process of the MATD3 Algorithm Model

**Algorithm 1** MATD3 on Air Combat Maneuvering

**for** agent i=1 to n **do**
  Initialize critic networks $Q_{\theta_{i_1}}$, $Q_{\theta_{i_2}}$, and actor network $\mu_i$ with random parameters $\theta_{i_1}$, $\theta_{i_2}$, $\theta^{\mu}$
  Initial target networks $\theta'_{i_1} \leftarrow \theta_{i_1}$, $\theta'_{i_2} \leftarrow \theta_{i_2}$, $\mu'_i \leftarrow \mu_i$
  Initial replay buffer $\mathcal{D}_i$
**end for**
**for** episode=1 to M **do**
  **for** t=1 to T **do**
    Select action with exploration noise $a \sim \pi_{\phi}(s) + \epsilon$,
  **end for**
  $\epsilon \in \mathcal{N}(0, \sigma)$ and observe reward $r$ and new state $s'$
  Store transition tuple $(s, a, r, s')$ in $\mathcal{D}_i$
  **for** agent i=1 to n **do**
    Sample mini-batch of $N$ transitions $(s, a, r, s')$ from $\mathcal{D}_i$
    $\tilde{a} \leftarrow \pi_{\phi'}(s') + \epsilon, \epsilon \sim clip(\mathcal{N}(0, \tilde{\sigma}), -c, c)$
    $y \leftarrow r + \gamma min_{i=1,2} Q_{\theta'_i}(s', \tilde{a})$
    Update critics $\theta_i \leftarrow argmin_{\theta_i} N^{-1} \sum (y - Q_{\theta_i}(s, a))^2$
    **if** t mod d **then**
      Update $\phi$ by the deterministic policy gradient:
      $\nabla_{\theta_i^{\mu}} J = N^{-1} \sum_j \nabla_{\theta_i^{\mu}} \mu_i(o_i^j) \times$
      $\nabla_{a_i Q_{\theta_{i_1}}(x^j, a^j, r^j, x'^j)|_{a_i = \mu_i(o_i^j)}}$
      Update target networks:
      $\theta'_i \leftarrow \tau\theta_i + (1-\tau)\theta'_i$
      $\theta'_i \leftarrow \tau\theta_i + (1-\tau)\theta'_i$
    **end if**
  **end for**
**end for**

An improved method for processing replay buffers in variable-length trajectory scenarios is shown in Fig. 4.

Each agent generates its own experience replay buffer based on its individual observation and action states, instead of sharing a common experience replay buffer. Furthermore, a logical flag parameter $m$ is added to the tuple $(s, a, r, s')$, which corresponds to different agents. If and only if $m_i > 0$, tuple $(s, a, r, s', m)$ is selected to compose the experience replay of the corresponding intelligent agent. During the training process of multi-agent reinforcement learning, each agent only interacts with data from its corresponding experience replay buffer. Therefore, the multi-buffer MATD3 algorithm (MBMATD3) is proposed in this paper. This improved MATD3 algorithm model with multi-agent selective experience replay method effectively reduces the instability in the training process of multi-agent reinforcement learning in non-uniform sequence scenarios and enhances the perfor-


Authorized licensed use limited to: ULAKBIM UASL - YILDIZ TEKNIK UNIVERSITESI. Downloaded on November 05,2026 at 10:37:09 UTC from IEEE Xplore. Restrictions apply.

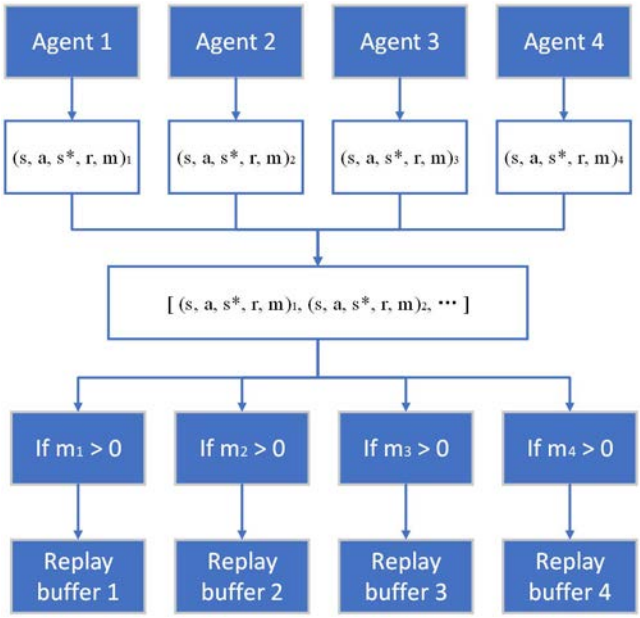



Fig. 4: Multi-agent Selective Experience Replay Method

mance of the fighter agents.

### 4.3 Reward Function

The overall reward function is composed of multiple terms including five different reward functions. Angle reward is defined as follows:

$$R_{i_{angle}} = \frac{k_{i_{angle}}}{|\phi_{ij}| + b_{i_{angle}}} - \frac{k_{i_{angle}}}{-|\phi_{ij}| + b_{i_{angle}} + \pi}, \quad (5)$$

where $\phi_{ij}$ represents the sight angles between our fighter agents and the enemy fighter agents. $R_{i_{angle}}$ represents the angle reward of each fighter agent on our side. $k_{i_{angle}}$ and $b_{i_{angle}}$ respectively represent the coefficient and the bias of the angle reward.

Height reward is defined as follows:

$$R_{i_{altitude}} = \begin{cases} k_{i_{altitude}} \times \dfrac{h_i - h_d}{h_d}, h_i < h_d, \\ 0, else, \end{cases} \quad (6)$$

where $R_{i_{altitude}}$ represents the altitude reward of each fighter agent on our side. $k_{i_{altitude}}$ and $h_d$ represent the coefficient and the altitude limit of the altitude reward, and $h_i$ represents the height of each fighter agent on our side.

Velocity reward is defined as follows:

$$R_{i_{velocity}} = \begin{cases} k_{i_{velocity}} \times \dfrac{v_i - v_d}{v_d}, v_i < v_d, \\ 0, else, \end{cases} \quad (7)$$

where $R_{i_{velocity}}$ represents the velocity reward of each fighter agent on our side. $k_{i_{velocity}}$ and $v_d$ respectively represent the coefficient and the velocity limit of the velocity reward, while $v_i$ represents the velocity of each fighter agent on our side.

Time reward is defined as follows:

$$R_{i_{time}} = r_{time}, \quad (8)$$

where $R_{i_{time}}$ represents the time reward of each fighter agent on our side. $r_{time}$ represents the value of the time reward.

Result reward is defined as follows:

$$R_{i_{result}} = \begin{cases} -r_{result}, destroyed, \\ 0, survive, \end{cases} \quad (9)$$

where $R_{i_{result}}$ represents the result reward of each fighter agent on our side. $-r_{result}$ represents the negative value of the time reward when our fighter agents are destroyed.

During air combat process, state variables such as altitude, velocity, angles, and flight time can be used to intuitively evaluate the advantage gained by our aircraft. Furthermore, these variables are highly correlated with the result of the air combat battle. Among these variables, the angles are the most important and have the greatest influence on the combat situation. If our aircraft and the enemy aircraft are flying head-on, both aircraft are considered to have the same advantage in terms of the angle advantage. However, if our aircraft is tailing and flying toward the enemy aircraft, they will get more advantages than the enemies.

If the velocity or altitude of one aircraft is getting lower, its probability of crashing and stalling will increase, which will make it lose advantages. On the other hand, if the velocity or altitude of one aircraft is getting higher, its missile attacking range will increase, and it will also get more kinetic energy to evade missiles, making it gain more advantage. Furthermore, in order to encourage the aircraft agent to explore more policy data, a time reward is added at every frame at each step when the agent interacts with the environment.

Finally, the overall reward is defined as:

$$R_{i_{total}} = R_{i_{angle}} + R_{i_{altitude}} + R_{i_{velocity}} + R_{i_{time}} + R_{i_{result}}, \quad (10)$$

where $R_{i_{total}}$ represents the overall reward of each fighter agent on our side.

## 5 Experiment Results

### 5.1 Air Combat Simulation

For every fighter, there is a height limit of 30,000 ft. If any fighter comes below 0ft or above 30,000 ft, it is considered crash and destroyed. The purpose of all fighter agents on each side is to search and destroy the opponent's fighter agents and to achieve the winner of the air combat battle simulation. Both enemy and our fighters spawn above the sea level at an altitude of 20,000 ft. And both sides start the engagement with a BVR distance of 90 km. Every fighter is equipped with two beyond-visual-range air-to-air AIM-120C missiles, with a maximum attacking range of 80 km. All fighters are equipped with a fixed missile launch strategy. When an opponent's fighter enters the no-escape zone of the missiles, the fighter locks onto the target and launches the missile. The time limit of simulated air combat battles is 300 seconds. When the time limit is reached, the side with more fighters alive is considered the winner. If fighters


Authorized licensed use limited to: ULAKBIM UASL - YILDIZ TEKNIK UNIVERSITESI. Downloaded on November 05,2026 at 10:37:09 UTC from IEEE Xplore. Restrictions apply.

alive on both sides are the same, the air combat is considered a draw. The air combat battle is simulated on DCS World (Digital Combat Simulator World), a combat flight simulation game developed primarily by Eagle Dynamics and The Fighter Collection.

### 5.2 Training Effect of the Improved Algorithm Model

The autonomous decision-making flight control models based on the MATD3 algorithm are constructed using PyTorch and Python environment. The size of every replay buffer is set to 100,000, and the learning rate of critic and actor networks of the MATD3 algorithm model is set to $5 \times 10^{-5}$. Reward function values of the original MATD3 algorithm and the MBMATD3 algorithm in air combat simulation battles are shown in Fig. 5. The rewards of MBMATD3 algorithm agents increase after 300 episodes of the training process. The trained MBMATD3 and MATD3 algorithm models of 290 episodes are chosen as the test model.




Fig. 5: Results on the cumulative reward

### 5.3 Experiment Analysis

Multi-agent air combat fighter agents with the test MBMATD3 and MATD3 models are used to fight with enemy fighters in the DCS world. The enemy fighters are equipped with built-in scripts 'Ace' (Top-level AI) and 'Experienced' (High-level AI). The outcome of 100 air combat battles is presented in Table 2.

Table 2: The Result of 100 Battles

| DCS World built-in script | total | victory | lose/draw |
|---|---|---|---|
| MBMATD3 vs 'Ace' | 100 | 81 | 19 |
| MBMATD3 vs 'Experienced' | 100 | 85 | 15 |
| MATD3 vs 'Ace' | 100 | 12 | 88 |
| MATD3 vs 'Experienced' | 100 | 29 | 71 |

Therefore, the MBMATD3 algorithm model with multi-agent selective experience replay method is effective in four-to-four BVR air combat battle simulations in DCS World. To describe the decision-making process of the MBMATD3 algorithm model, the battle is replayed and visualized using Tacview, an air combat visualization software. After the training process, our fighter agents are capable of making different combat decisions in various battlefield states, such as the rapid descent maneuver strategy, the spiral maneuver strategy, the low-altitude hovering maneuver strategy, the coordinated attack strategy, etc. These combat decisions are shown in Fig. 6, Fig. 7, Fig. 8, and Fig. 9, in which the blue fighters and missiles belong to our side, and the red fighters and missiles belong to the enemy side.

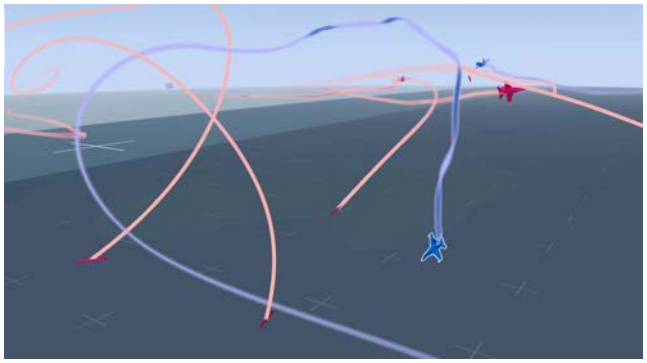


Fig. 6: Rapid descent maneuver

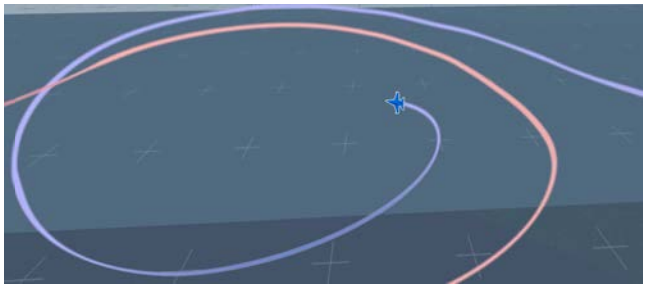


Fig. 7: Spiral maneuver

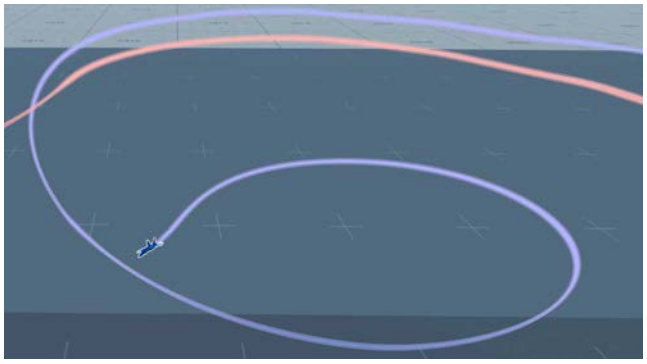


Fig. 8: Low-altitude hovering maneuver

In Fig. 6, when the enemy missiles possess a significant threat and our fighters possess a considerable altitude and speed advantage, the rapid descent maneuver strategy is selected by fighter agents to deplete the kinetic energy of the enemy missiles, ultimately evading them. In Fig. 7, when our fighter agents detect that an enemy missile is launched from a distant position, the spiral maneuver strategy is selected to disrupt the tracking direction of the enemy missile, in order to increase the probability of evasion. In the situation of Fig. 8 where our fighters expand all missiles and are currently not under immediate threat, the low-altitude hovering maneuver strategy is selected to prevent being locked


Authorized licensed use limited to: ULAKBIM UASL - YILDIZ TEKNIK UNIVERSITESI. Downloaded on November 05,2026 at 10:37:09 UTC from IEEE Xplore. Restrictions apply.

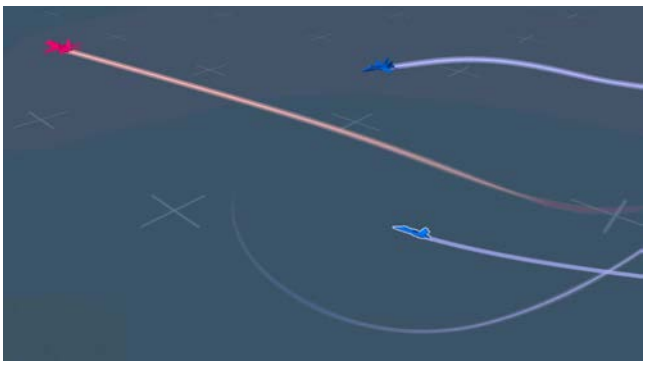


Fig. 9: Coordinated attack strategy

onto by the radars of the enemy missiles. In Fig. 9, after evading enemy missiles, our fighter agents adopt the coordinated attack strategy to eliminate enemy fighters and finally win the simulated air combat battle.

## 6 Conclusions

In this paper, an autonomous fighter decision-making model based on the MBMATD3 algorithm with the multi-agent selective experience replay method has realized the intelligent decision-making of four-to-four multi-agent BVR air combat battle simulation and has achieved a high winning rate. The MBMATD3 algorithm with the multi-agent selective experience replay method is effective at decisions of continuous action in multi-agent simulated air combat. After a well-considered reward function is designed, the algorithmic model can effectively explore various evasion maneuvers, and it can also achieve better performance in BVR air combat. Multi-agent selective experience replay method is able to increase the training stability of the algorithm, and effectively enhance the performance. Fighter agents equipped with the MATD3 algorithm with improved experience replay method perform better than built-in Top-level AI script in DCS World and achieve a higher winning rate in simulated air combat experiments.

## References


[1] Z. Chang, H. Deng, L. You, G. Min, S. Garg, and G. Kaddoum, Trajectory design and resource allocation for multi-UAV networks: Deep reinforcement learning approaches, *IEEE Trans. on Network Science and Engineering*, 10(5): 2940–2951, 2023.

[2] C. Huang, K. Dong, H. Huang, S. Tang, and Z. Zhang, Autonomous air combat maneuver decision using Bayesian inference and moving horizon optimization, *Journal of Systems Engineering and Electronics*, 29(1): 86–97, 2018.

[3] A. Stoica, E. Dente, Y. Iwashita, and A. Chiolerio, UAVs you can't see or hear - a survey of key technologies, in *2015 Sixth International Conference on Emerging Security Technologies (EST)*, 2015: 50–55.

[4] N. C. Luong, D. T. Hoang, S. Gong, D. Niyato, P. Wang, Y.-C. Liang, and D. I. Kim, Applications of deep reinforcement learning in communications and networking: A survey, *IEEE Communications Surveys & Tutorials*, 21(4): 3133–3174, 2019.

[5] J. Zhang, Q. Yang, G. Shi, Y. Lu, and Y. Wu, UAV cooperative air combat maneuver decision based on multi-agent reinforcement learning, *Journal of Systems Engineering and Electronics*, 32(6): 1421–1438, 2021.

[6] M. E. Mkiramweni, C. Yang, J. Li, and W. Zhang, A survey of game theory in unmanned aerial vehicles communications, *IEEE Communications Surveys & Tutorials*, 21(4): 3386–3416, 2019.

[7] Y. Ma, G. Wang, X. Hu, H. Luo, and X. Lei, Cooperative occupancy decision making of multi-UAV in beyond-visual-range air combat: A game theory approach, *IEEE Access*, 8: 11624–11634, 2020.

[8] M. Asim, W. K. Mashwani, S. B. Belhaouari, and S. Hassan, A novel genetic trajectory planning algorithm with variable population size for multi-UAV-assisted mobile edge computing system, *IEEE Access*, 9: 125569–125579, 2021.

[9] X. Wen, Y. Ruan, Y. Li, H. Xia, R. Zhang, C. Wang, W. Liu, and X. Jiang, Improved genetic algorithm based 3-D deployment of UAVs, *Journal of Communications and Networks*, 24(2): 223–231, 2022.

[10] P. Du and H. Liu, Study on air combat tactics decision-making based on bayesian networks, in *2010 2nd IEEE International Conference on Information Management and Engineering*, 2010: 252–256.

[11] G. Hu, C. Zhou, X. Zhang, H. Zhang, Z. Song, and Z. Zhou, A neural network-based intelligent decision-making in the air-offensive campaign with simulation, in *2020 16th International Conference on Computational Intelligence and Security (CIS)*, 2020: 342–346.

[12] C. Liu, J. Xu, J. Zhang, and L. Yang, Study of the vertical pursuit point design for TTP-based air combat autonomous decision-making, in *2023 42nd Chinese Control Conference (CCC)*, 2023: 3042–3048.

[13] O. S. Oubbati, M. Atiquzzaman, A. Baz, H. Alhakami, and J. Ben-Othman, Dispatch of UAVs for urban vehicular networks: A deep reinforcement learning approach, *IEEE Trans. on Vehicular Technology*, 70(12): 13174–13189, 2021.

[14] G. Shen, L. Lei, Z. Li, S. Cai, L. Zhang, P. Cao, and X. Liu, Deep reinforcement learning for flocking motion of multi-UAV systems: Learn from a digital twin, *IEEE Internet of Things Journal*, 9(13): 11141–11153, 2022.

[15] A. P. Pope, J. S. Ide, D. Mićović, H. Diaz, J. C. Twedt, K. Alcedo, T. T. Walker, D. Rosenbluth, L. Ritholtz, and D. Javorsek, Hierarchical reinforcement learning for air combat at DARPA's AlphaDogfight trials, *IEEE Trans. on Artificial Intelligence*, 4(6): 1371–1385, 2023.

[16] S. Fujimoto, H. van Hoof, and D. Meger, Addressing function approximation error in actor-critic methods, in *Proceedings of the 35th International Conference on Machine Learning*, volume 80, 2018: 1587–1596.

[17] R. Lowe, Y. Wu, A. Tamar, J. Harb, O. Pieter Abbeel, and I. Mordatch, Multi-agent actor-critic for mixed cooperative-competitive environments, in *Advances in Neural Information Processing Systems*, volume 30, 2017.

[18] T. Lilicrap, J. Hunt, A. Pritzel, N. Hess, T. Erez, D. Silver, Y. Tassa, and D. Wiestra, Continuous control with deep reinforcement learning, in *International Conference on Representation Learning (ICRL)*, 2016.

[19] H. Van Hasselt, A. Guez, and D. Silver, Deep reinforcement learning with double Q-learning, in *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 30, 2016: 2094–2100.

[20] B. Zhang, Z. Chen, X. Wu, D. Cao, and W. Hu, A MATD3-based voltage control strategy for distribution networks considering active and reactive power adjustment costs, in *2022 IEEE International Conference on Power Systems and Electrical Technology (PSET)*, 2022: 189–194.




Authorized licensed use limited to: ULAKBIM UASL - YILDIZ TEKNIK UNIVERSITESI. Downloaded on November 05,2026 at 10:37:09 UTC from IEEE Xplore. Restrictions apply.